

A Demonstration Project for Rule-Based and AI-Assisted Fault Diagnosis in a GNS3 Lab

Author: *Nabin tharu*

Date: 9/30/2026

1. Executive Summary

This report describes **Netmon**, a Python-based network monitoring and root-cause analysis (RCA) tool deployed in a GNS3 lab. Netmon polls Cisco routers and switches every 20 seconds over SNMP and ICMP, evaluates the collected metrics against a rule-based detection engine, enriches each detected incident with an AI-generated diagnosis via the Groq API, and delivers evidence-packaged email alerts routed to the correct operational team. Gateway redundancy in the lab network is provided by VRRP, and device-level redundancy is inferred from multiple independent paths, allowing the tool to distinguish a device power failure from a single-link cable fault.

The project is intended as a demonstration of how classical rule-based RCA can be combined with large-language-model reasoning to produce alerts that are diagnostically rich.

2. Objectives

- Monitor a small multi-router lab continuously over ICMP and SNMP v2c.
- Detect common fault classes device offline, interface down, routing failure, reboot, latency anomalies with confidence scoring.
- Attach a compact evidence package to every alert so the receiving engineer can see what happened, not just that something happened.
- Route each alert to a role-appropriate team based on the cause category.
- Provide high-availability gateway access to the monitoring host without installing routing services on the host itself.

3. Lab Network Design

3.1 Topology Overview

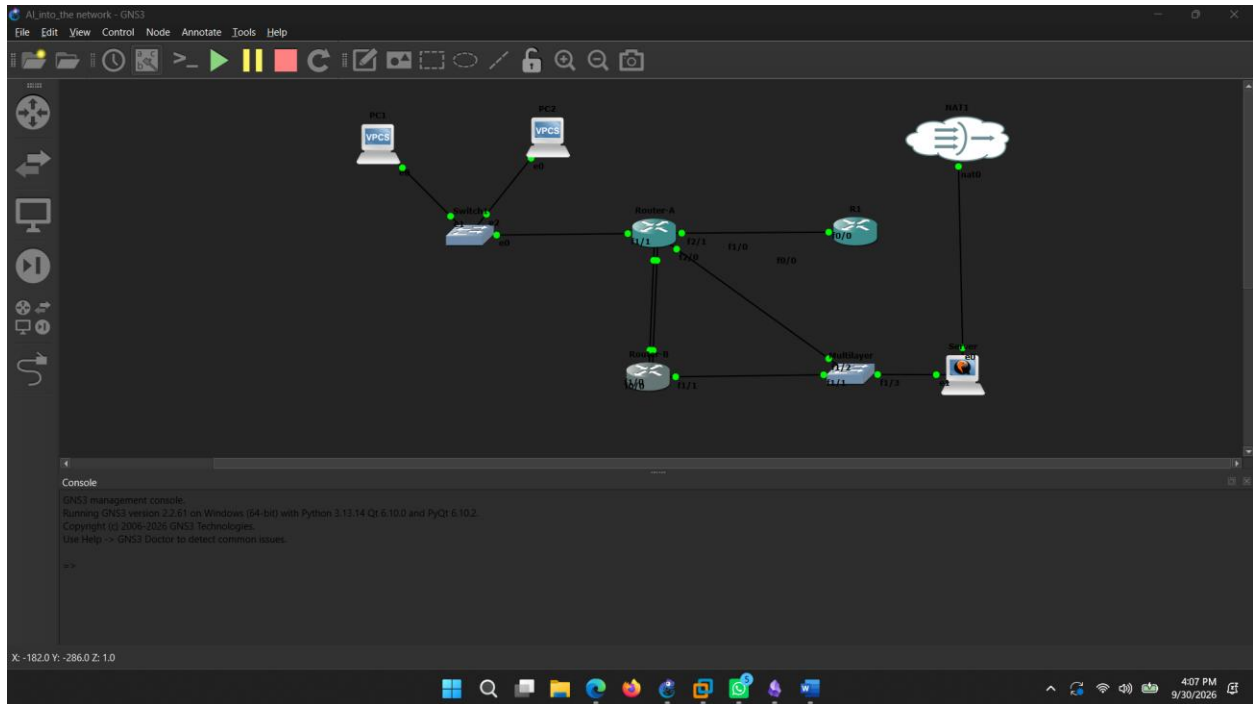


Figure 1- Network Topology Diagram

The lab consists of five network nodes and a NAT gateway:

Device	Role	Key Interfaces
R1	Cisco c7200 edge router	Fa0/0 = 10.0.3.1
Router-A	Cisco c7200 core router (VRRP Master)	Fa0/0 = 10.0.2.1, Fa1/0 = 10.0.1.1, Fa2/0 = 192.168.2.1, Fa2/1 = 10.0.3.2
Router-B	Cisco c7200 access router (VRRP Backup)	Fa0/0 = 10.0.2.2, Fa1/0 = 10.0.1.2, Fa1/1 = 192.168.2.2
Switch1	Cisco c2691 with NM-16ESW module	VLAN 1 SVI = 192.168.2.10
Ubuntu Server	Monitoring host	ens3 = 192.168.214.140 (NAT), ens4 = 192.168.2.200 (LAN)

NAT1	GNS3 NAT node	Internet gateway, 192.168.214.2
VRRP VIP	Virtual gateway owned by Router-A/B	192.168.2.254

3.2 Design Principles

Dual homing on the monitoring host. The Ubuntu host uses two network interfaces: ens3 for Internet access via the GNS3 NAT node, and ens4 on the lab LAN for polling devices. Internet traffic and lab traffic use physically distinct paths, so heavy polling never degrades the tool's own ability to reach the AI service or the mail server.

First-hop redundancy with VRRP. Router-A and Router-B form a VRRP group on the lab LAN. Router-A holds priority 120 and is the master; Router-B holds priority 100 and is the backup. Both preempt. The virtual IP 192.168.2.254 is owned by whichever router is currently master. The monitoring host uses the VIP as its only gateway for the 10.0.0.0/8 network, so gateway failover is transparent to the host no host configuration changes when the active roster changes.

Static routes at the leaves. R1 has a static route back to the LAN segment (ip route 192.168.2.0 255.255.255.0 10.0.3.2). Router-B has a static route to R1 (ip route 10.0.3.0 255.255.255.252 10.0.2.1). No dynamic routing protocol is used between routers, keeping the design deterministic and easy to reason about.

Redundant paths to the monitoring host. Router-B is reachable from the monitoring host through three independent addresses:

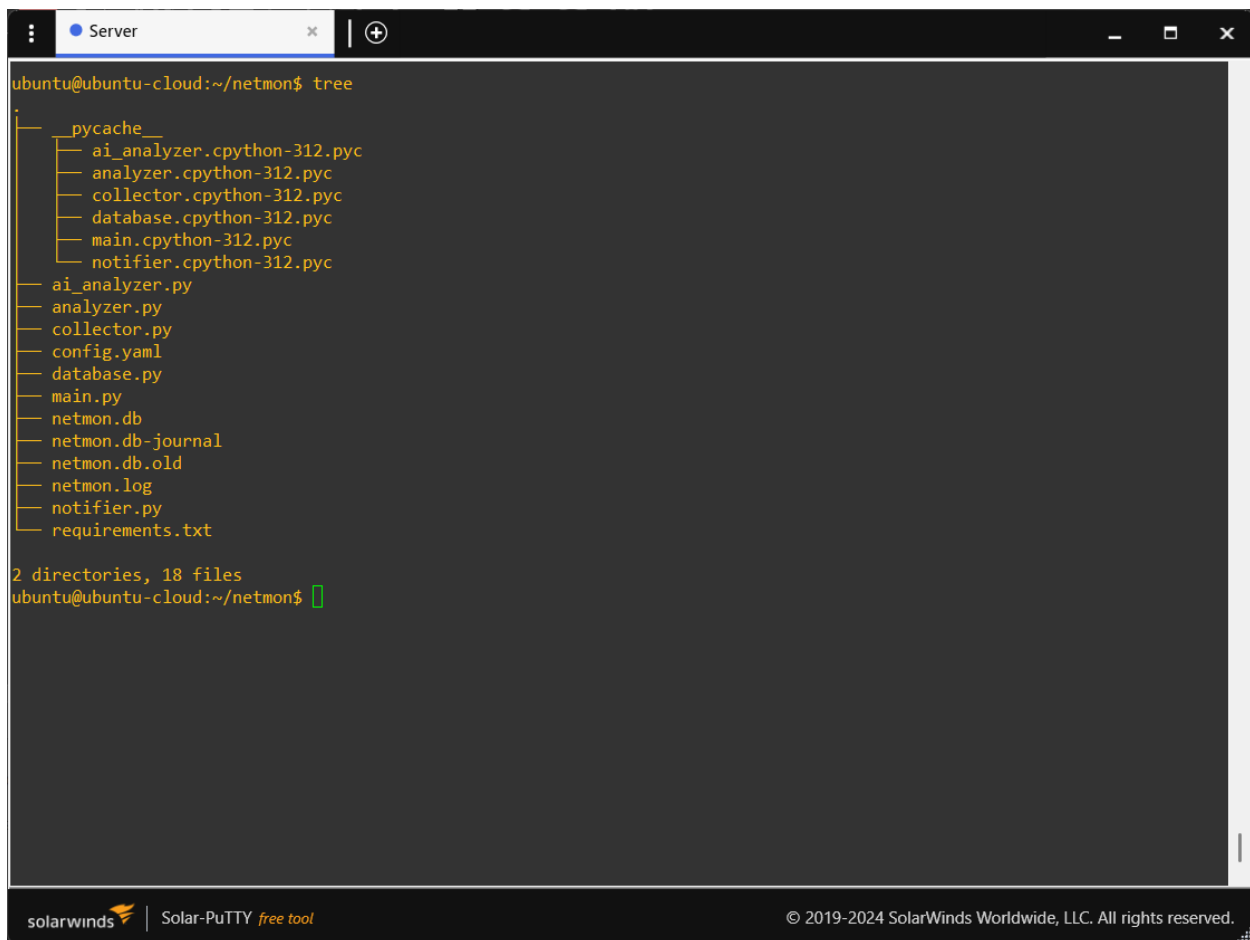
- 192.168.2.2 direct on the LAN via Switch1
- 10.0.2.2 via Router-A over the 10.0.2.0/30 link
- 10.0.1.2 via Router-A over the 10.0.1.0/30 link

This redundancy is not just for resilience it is also the mechanism the tool uses to determine whether an outage is device-level or path-level. If all three addresses fail in the same cycle, no single cable or port fault can explain it; the device itself must be down.

MTU tuning on the NAT interface. The NAT-facing interface uses MTU 1400 to avoid fragmentation issues introduced by the emulated NAT path in GNS3, ensuring stable HTTPS sessions to the AI API and SMTP server.

4. Tool Architecture

Netmon is written in Python 3.12 and consists of six modules. Data flows linearly: **collect** → **analyze** → **correlate** → **persist** → **notify**.



```
ubuntu@ubuntu-cloud:~/netmon$ tree
.
├── pycache
│   ├── ai_analyzer.cpython-312.pyc
│   ├── analyzer.cpython-312.pyc
│   ├── collector.cpython-312.pyc
│   ├── database.cpython-312.pyc
│   ├── main.cpython-312.pyc
│   └── notifier.cpython-312.pyc
├── ai_analyzer.py
├── analyzer.py
├── collector.py
├── config.yaml
├── database.py
├── main.py
├── netmon.db
├── netmon.db-journal
├── netmon.db.old
├── netmon.log
├── notifier.py
└── requirements.txt

2 directories, 18 files
ubuntu@ubuntu-cloud:~/netmon$
```

The screenshot shows a terminal window titled "Server" with a tab icon and a "+" button. The terminal output displays the directory tree of the netmon project. The files listed are: pycache (containing pyc files for ai_analyzer, analyzer, collector, database, main, and notifier), ai_analyzer.py, analyzer.py, collector.py, config.yaml, database.py, main.py, netmon.db, netmon.db-journal, netmon.db.old, netmon.log, notifier.py, and requirements.txt. A summary line indicates "2 directories, 18 files". The terminal prompt is "ubuntu@ubuntu-cloud:~/netmon\$". At the bottom of the terminal window, there is a footer with the SolarWinds logo, "Solar-PuTTY free tool", and a copyright notice: "© 2019-2024 SolarWinds Worldwide, LLC. All rights reserved."

Module	Responsibility
collector.py	ICMP ping (primary and alternate addresses), SNMP v2c walks via pysnmp 7.x asyncio API
analyzer.py	Rule-based fault detection, cross-device correlation, AI enrichment
ai_analyzer.py	Groq LLM integration for second-opinion diagnosis
notifier.py	SMTP email delivery, per-cause team routing, 10-second send timeout
database.py	SQLite store for time-series metrics and incidents
main.py	Parallel orchestration loop with colorized per-cycle output

5. How the Tool Works

5.1 Collection

Every 20 seconds, the tool polls all configured devices in parallel using a thread pool. For each device it performs:

- **ICMP ping** to the primary address and every alternate address. The device is only marked unreachable when *every* address fails.
- **SNMP v2c walks** for the interface table, error and discard counters, system uptime, Cisco memory pool usage, and where the hardware exposes its power supply state via ciscoEnvMonSupplyState.

Ping uses `-c 2 -W 1` for a maximum of two seconds per address. SNMP walks use a one-second timeout so unresponsive OIDs fail quickly without stalling the loop.

5.2 Rule-Based Detection

The analyzer compares each snapshot against the previous one and evaluates a fixed rule set. Rules operate on **transitions**, not absolute values, so steady-state faults do not repeatedly fire.

Rule	Condition	Cause	Base Confidence
Interface down + ping fail	ping fails and at least one interface reports down	cabling	90%
Ping fail, SNMP alive	ping fails but SNMP still responds	routing	75%
Multi-path device offline	all alternate addresses fail in the same cycle	power	95%
Partial path failure	some addresses fail, others respond	cabling	80%
Single-path device offline	the only known address fails	cabling	75%
Interface down while device reachable	an interface transitions up→down	cabling	75%
Reboot	uptime decreases between readings	reboot	95%
Memory pressure	memory usage $\geq$ 85%	memory	70%
Error spike	error + discard delta $\geq$ 50 per cycle	cabling	80%
Latency anomaly	average RTT $\geq$ 400 ms	routing	60%

5.3 Multi-Path Reachability and Device-Offline Detection

When a device is unreachable, the analyzer consults the `reachability_detail` field produced by the collector, which lists every address the device was tested at and whether each responded.

- **All paths failed, more than one path exists** → the device is unreachable on physically distinct paths simultaneously. No single cable fault can cause this. The incident is classified as power and routed to the facilities team.
- **Some paths failed** → the device is still alive; a specific link is broken. The incident is classified as cabling and includes the failing address.
- **Only one path exists and it failed** → the tool cannot prove the device itself is down. The incident is classified as cabling with a message stating that ping and SNMP both failed, and a wire or configuration issue is possible.

This logic gives the tool a way to distinguish device-level from link-level faults without relying on hardware sensors that emulated devices do not expose.

5.4 AI-Assisted Diagnosis

After a rule fires, the incident and its evidence package the last two readings, the interface state history, and the triggering rule are sent to **Groq** running openai/gpt-oss-20b. The model is asked to act as a principal network engineer and return a JSON object with `cause_category`, `confidence`, `severity`, and `explanation`.

The AI's response is used to **refine** the rule-based output when the AI's cause category agrees with the rule. If the AI disagrees, the rule's short, deterministic explanation is preserved. This keeps the alert readable and consistent while still benefiting from the LLM's pattern recognition on ambiguous cases.

If the AI is unreachable network issue, rate limit, or model unavailable the incident still fires with the rule's own explanation and is marked `ai_verified: false`. Detection never depends on AI availability.

5.5 Evidence Packaging

Every incident carries a compact evidence package:

- The triggering rule identifier
- Thresholds in effect at the time
- The last two readings labeled **BEFORE** and **DURING** showing reachability, latency, packet loss, memory, uptime, and per-interface status
- The AI verdict and verification flag

Each metric in the email is rendered explicitly, either as a value or as N/A (not supported by device) when the hardware does not expose it. Nothing is fabricated.

5.6 Alert Routing and Notification

Alerts are delivered by Gmail SMTP using an App Password stored in the environment. Each incident's cause maps to a receiving team:

Cause	Team
cabling, reboot	NOC Team
cpu, memory, routing	Network Engineering
power, thermal	Facilities Team

Duplicate suppression prevents re-alerting for the same device and cause within 300 seconds. A ten-second SMTP timeout prevents a slow mail server from stalling the collection loop.

5.7 Console Output

The orchestration loop prints a color-coded status block every cycle:

```
Start time : 2026-09-30 10:36:55
Devices   : 4
Poll every : 20s
AI backend : Groq / openai/gpt-oss-20b
Email      : enabled
Database   : netmon.db
Mode       : parallel polling (4 workers)

CYCLE 1 • 10:36:55

● UP R1 ifaces: 6 latency: 84.1 ms mem: 13.0% uptime: 51716
  └─ DOWN: FastEthernet1/1, FastEthernet2/0, FastEthernet2/1
● UP Router-A ifaces: 6 latency: 35.9 ms mem: 13.0% uptime: 51792

  △ INCIDENT #17
    Cause      : reboot
    Severity    : Critical
    Confidence  : 95%
    Device     : Router-A
    Rule       : uptime decreased
    Explanation: Router-A: uptime reset from 277746 to 51792. Conclusion: reboot/power loss/crash.
                  AI verified: no (rule-based only)
                  ✓ Email alert sent
● UP Switch1 ifaces: 20 latency: 45.3 ms mem: 11.3% uptime: 90536
  └─ DOWN: FastEthernet0/0, FastEthernet0/1, FastEthernet1/0, FastEthernet1/4 (+11 more)

  △ INCIDENT #18
    Cause      : reboot
    Severity    : Critical
    Confidence  : 95%
    Device     : Switch1
    Rule       : uptime decreased
    Explanation:
```

Figure 2 interface

6. Demonstration Scenario

The tool is designed to be demonstrated by simulating real faults in GNS3.

Cable cut on R1. Shut down FastEthernet0/0 on R1 from its console. Within one cycle, the tool reports ● DOWN R1 with an incident classified as cabling and a one-line explanation reading *"R1: unreachable due to ping and SNMP failure. Possible wire or configuration issue."* The alert is routed to the NOC Team.

Power loss on Router-B. Stop the Router-B node in GNS3. All three independent paths to Router-B fail in the same cycle. The tool raises a power incident with 95% confidence, explained as *"Router-B is unreachable on all paths. Possible power loss or hardware failure."* The alert is routed to the Facilities Team.

Restore. Restart the router or no shutdown the interface. The next cycle returns to all-green, no incident fires.

7. Conclusion

Netmon demonstrates a complete monitoring and RCA pipeline built from open-source components. It polls devices in parallel over SNMP and ICMP, evaluates their state against a deterministic rule engine, uses multi-path reachability to distinguish device-level from link-level faults, and enriches each alert with an AI-generated diagnosis when the AI agrees with the rule engine. Every incident carries a compact evidence package so the receiving team can see what happened rather than just that something happened. Alerts are routed by cause to the correct team, and gateway redundancy is provided by VRRP without installing any routing services on the monitoring host.

The result is a tool that is fast, honest about what it does and does not know, and produces alerts an on-call engineer can act on.

Appendix A Network Configuration Commands

The following commands were applied to build the lab network. They are grouped by device.

A.1 R1 — Cisco c7200 Edge Router

enable

configure terminal

hostname R1

interface FastEthernet0/0

```
ip address 10.0.3.1 255.255.255.252
no shutdown
Return route to the monitoring LAN
ip route 192.168.2.0 255.255.255.0 10.0.3.2
snmp-server community public RO
end
write memory
```

A.2 Router-A — Cisco c7200 Core Router (VRRP Master)

```
enable
configure terminal
hostname Router-A
interface FastEthernet0/0
ip address 10.0.2.1 255.255.255.252
no shutdown
interface FastEthernet1/0
ip address 10.0.1.1 255.255.255.252
no shutdown
interface FastEthernet2/0
ip address 192.168.2.1 255.255.255.0
vrrp 1 ip 192.168.2.254
vrrp 1 priority 120
vrrp 1 preempt
no shutdown

interface FastEthernet2/1
ip address 10.0.3.2 255.255.255.252
```

```
no shutdown
snmp-server community public RO
end
write memory
```

A.3 Router-B — Cisco c7200 Access Router (VRRP Backup)

```
enable
configure terminal
hostname Router-B
interface FastEthernet0/0
ip address 10.0.2.2 255.255.255.252
no shutdown
interface FastEthernet1/0
ip address 10.0.1.2 255.255.255.252
no shutdown
interface FastEthernet1/1
ip address 192.168.2.2 255.255.255.0
vrrp 1 ip 192.168.2.254
vrrp 1 preempt
no shutdown
Static route to R1 through Router-A
ip route 10.0.3.0 255.255.255.252 10.0.2.1
snmp-server community public RO
end
write memory
```

A.4 Switch1 — Cisco c2691 with NM-16ESW

```
enable
```

```
configure terminal
```

```
hostname Switch1
```

```
interface Vlan1
```

```
ip address 192.168.2.10 255.255.255.0
```

```
no shutdown
```

```
ip default-gateway 192.168.2.254
```

```
snmp-server community public RO
```

```
end
```

```
write memory
```

A.5 Ubuntu — Netplan Configuration

```
/etc/netplan/01-netcfg.yaml
```

```
yaml
```

```
network:
```

```
  version: 2
```

```
  ethernets:
```

```
    ens3:
```

```
      dhcp4: true
```

```
      mtu: 1400
```

```
/etc/netplan/02-lab.yaml
```

```
yaml
```

```
network:
```

```
  version: 2
```

```
  ethernets:
```

ens4:

dhcp4: no

addresses:

- 192.168.2.200/24

routes:

- to: 10.0.0.0/8

via: 192.168.2.254

nameservers:

addresses: [1.1.1.1, 8.8.8.8]

Apply:

bash

sudo netplan apply

Appendix B — Tool Configuration

B.1 config.yaml

yaml

poll_interval: 20

history_window: 5

db_path: "netmon.db"

log_path: "netmon.log"

devices:

- name: "R1"

host: "10.0.3.1"

alt_hosts: []

type: "router"

```
snmp_community: "public"
snmp_version: 2
- name: "Router-A"
  host: "10.0.1.1"
  alt_hosts: ["192.168.2.1"]
  type: "router"
  snmp_community: "public"
  snmp_version: 2
- name: "Router-B"
  host: "192.168.2.2"
  alt_hosts: ["10.0.2.2", "10.0.1.2"]
  type: "router"
  snmp_community: "public"
  snmp_version: 2
- name: "Switch1"
  host: "192.168.2.10"
  alt_hosts: []
  type: "switch"
  snmp_community: "public"
  snmp_version: 2
```

oids:

```
sysUpTime:    "1.3.6.1.2.1.1.3.0"
ifDescr:      "1.3.6.1.2.1.2.2.1.2"
ifOperStatus: "1.3.6.1.2.1.2.2.1.8"
ifInErrors:   "1.3.6.1.2.1.2.2.1.14"
```

ifOutErrors: "1.3.6.1.2.1.2.2.1.20"
ifInDiscards: "1.3.6.1.2.1.2.2.1.13"
ifOutDiscards: "1.3.6.1.2.1.2.2.1.19"
ifLastChange: "1.3.6.1.2.1.2.2.1.9"
ciscoMemoryPoolUsed: "1.3.6.1.4.1.9.9.48.1.1.1.5"
ciscoMemoryPoolFree: "1.3.6.1.4.1.9.9.48.1.1.1.6"
ciscoEnvMonSupplyState: "1.3.6.1.4.1.9.9.13.1.5.1.3"

thresholds:

cpu_sustained_pct: 90
cpu_sustained_cycles: 6
mem_used_pct: 85
error_delta_threshold: 50
ping_latency_crit_ms: 400

groq:

enabled: true
model: "openai/gpt-oss-20b"
timeout_seconds: 15

notifications:

email:
enabled: true
smtp_host: "smtp.gmail.com"
smtp_port: 465
use_ssl: true

username: "<sender>@gmail.com"

password_env_var: "GMAIL_APP_PASSWORD"

from_addr: "<sender>@gmail.com"

teams:

cabling: { name: "NOC Team", email: "<noc>+noc@gmail.com" }

routing: { name: "Net Eng Team", email: "<noc>+neteng@gmail.com" }

cpu: { name: "Net Eng Team", email: "<noc>+neteng@gmail.com" }

memory: { name: "Net Eng Team", email: "<noc>+neteng@gmail.com" }

reboot: { name: "NOC Team", email: "<noc>+noc@gmail.com" }

power: { name: "Facilities Team", email: "<noc>+facilities@gmail.com" }

thermal: { name: "Facilities Team", email: "<noc>+facilities@gmail.com" }

unknown: { name: "NOC Team", email: "<noc>+noc@gmail.com" }

B.2 Environment Variables

bash

export GROQ_API_KEY="gsk..."

export GMAIL_APP_PASSWORD="xxxx xxxx xxxx xxxx"

B.3 Systemd Unit — /etc/systemd/system/netmon.service

ini

[Unit]

Description=Network Monitoring & RCA

After=network-online.target

Wants=network-online.target

[Service]

Type=simple

User=ubuntu

WorkingDirectory=/home/ubuntu/netmon

EnvironmentFile=/etc/netmon/env

ExecStart=/home/ubuntu/netmon-venv/bin/python3 /home/ubuntu/netmon/main.py

Restart=on-failure

RestartSec=10

[Install]

WantedBy=multi-user.target

B.4 Installation

bash

sudo apt update

sudo apt install -y python3 python3-venv iputils-ping snmp

python3 -m venv ~/netmon-venv

source ~/netmon-venv/bin/activate

pip install --no-cache-dir pysnmp pyasn1 netmiko paramiko PyYAML requests groq openai

Appendix C — Source Code

C.1 database.py

```
python
```

```
import sqlite3, json
```

```
from datetime import datetime
```

```
class Database:
```

```
    def __init__(self, path="netmon.db"):
```

```
        self.path = path
```

```
        self._init_schema()
```

```
    def _conn(self):
```

```
        c = sqlite3.connect(self.path)
```

```
        c.row_factory = sqlite3.Row
```

```
        return c
```

```
    def _init_schema(self):
```

```
        with self._conn() as c:
```

```
            c.executescript("""
```

```
            CREATE TABLE IF NOT EXISTS metrics (
```

```
                id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
                ts TEXT NOT NULL, device TEXT NOT NULL,
```

```
                reachable INTEGER, latency_ms REAL, loss_pct REAL,
```

```
                uptime_ticks INTEGER, cpu_5min REAL, mem_used_pct REAL,
```

```
                temp_c REAL, power_ok INTEGER, interfaces_json TEXT
```

```
            );
```

```
            CREATE INDEX IF NOT EXISTS idx_metrics_dev_ts ON metrics(device, ts);
```

```
            CREATE TABLE IF NOT EXISTS incidents (
```

```

id INTEGER PRIMARY KEY AUTOINCREMENT,
ts TEXT NOT NULL, device TEXT NOT NULL,
severity TEXT, cause TEXT, confidence INTEGER,
explanation TEXT, evidence_json TEXT, notified INTEGER DEFAULT 0
);
"""

```

```

def insert_metric(self, snap):
    with self._conn() as c:
        c.execute("""INSERT INTO metrics
(ts, device, reachable, latency_ms, loss_pct, uptime_ticks,
cpu_5min, mem_used_pct, temp_c, power_ok, interfaces_json)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""", (
snap["ts"], snap["device"], snap.get("reachable"),
snap.get("latency_ms"), snap.get("loss_pct"),
snap.get("uptime_ticks"), snap.get("cpu_5min"),
snap.get("mem_used_pct"), snap.get("temp_c"),
snap.get("power_ok"), json.dumps(snap.get("interfaces", {}))))

```

```

def recent_metrics(self, device, n=5):
    with self._conn() as c:
        rows = c.execute("""SELECT * FROM metrics WHERE device=?
ORDER BY id DESC LIMIT ?""", (device, n)).fetchall()
    return [dict(r) for r in rows][::-1]

```

```

def insert_incident(self, inc):

```

```

with self._conn() as c:

    cur = c.execute("""INSERT INTO incidents

        (ts, device, severity, cause, confidence, explanation,

        evidence_json, notified) VALUES (?, ?, ?, ?, ?, ?, ?, 0)""", (

        inc["ts"], inc["device"], inc["severity"], inc["cause"],

        inc["confidence"], inc["explanation"], json.dumps(inc["evidence"])))

    return cur.lastrowid

```

```

def mark_notified(self, inc_id):

```

```

    with self._conn() as c:

        c.execute("UPDATE incidents SET notified=1 WHERE id=?", (inc_id,))

```

```

def recent_incident(self, device, cause, within_sec=300):

```

```

    cutoff = datetime.utcnow().timestamp() - within_sec

    with self._conn() as c:

        rows = c.execute("""SELECT * FROM incidents

            WHERE device=? AND cause=? ORDER BY id DESC LIMIT 5""",

            (device, cause)).fetchall()

```

```

    for r in rows:

```

```

        try:

```

```

            if datetime.fromisoformat(r["ts"]).timestamp() >= cutoff:

```

```

                return dict(r)

```

```

            except Exception:

```

```

                continue

```

```

    return None

```

C.2 collector.py

python

```
import subprocess, re, logging, warnings
```

```
from datetime import datetime
```

```
warnings.filterwarnings("ignore", message=".*TripleDES.*")
```

```
warnings.filterwarnings("ignore", category=UserWarning, module="paramiko")
```

```
log = logging.getLogger("collector")
```

```
def ping(host, count=2, timeout=1):
```

```
    try:
```

```
        out = subprocess.run(
```

```
            ["ping", "-c", str(count), "-W", str(timeout), host],
```

```
            capture_output=True, text=True, timeout=count * timeout + 2).stdout
```

```
    except Exception:
```

```
        return {"reachable": False, "latency_ms": None, "loss_pct": 100.0}
```

```
    loss_m = re.search(r"(\d+(?:\.\d+)?)% packet loss", out)
```

```
    loss = float(loss_m.group(1)) if loss_m else 100.0
```

```
    rtt_m = re.search(r"rtt min/avg/max/mdev = [\d.]+/([\d.]+)/", out)
```

```
    latency = float(rtt_m.group(1)) if rtt_m else None
```

```
    return {"reachable": loss < 100.0, "latency_ms": latency, "loss_pct": loss}
```

```
def _snmp_get(host, community, oids, version=2, timeout=1, retries=1):
```

```

import asyncio

from pysnmp.hlapi.v3arch.asyncio import (
    SnmpEngine, CommunityData, UdpTransportTarget,
    ContextData, ObjectType, ObjectIdentity, get_cmd)

async def _run():
    result = {}

    try:
        engine = SnmpEngine()

        target = await UdpTransportTarget.create(
            (host, 161), timeout=timeout, retries=retries)

        errInd, errStat, _, varbinds = await get_cmd(
            engine, CommunityData(community, mpModel=1 if version == 2 else 0),
            target, ContextData(),
            *[ObjectType(ObjectIdentity(o)) for o in oids])

        if errInd or errStat:
            return result

        for vb in varbinds:
            if vb[1] is None or "No Such" in str(vb[1]):
                continue

            try: result[str(vb[0])] = int(vb[1])

            except Exception: result[str(vb[0])] = str(vb[1])

        except Exception:
            pass

        return result

    try:
        return asyncio.run(_run())

```

```
except Exception:
```

```
    return {}
```

```
def _snmp_walk(host, community, base_oid, version=2, timeout=1):
```

```
    import asyncio
```

```
    from pysnmp.hlapi.v3arch.asyncio import (
```

```
        SnmpEngine, CommunityData, UdpTransportTarget,
```

```
        ContextData, ObjectType, ObjectIdentity, walk_cmd)
```

```
    async def _run():
```

```
        result = {}
```

```
        try:
```

```
            engine = SnmpEngine()
```

```
            target = await UdpTransportTarget.create(
```

```
                (host, 161), timeout=timeout, retries=1)
```

```
            async for errInd, errStat, _, varbinds in walk_cmd(
```

```
                engine, CommunityData(community, mpModel=1 if version == 2 else 0),
```

```
                target, ContextData(),
```

```
                ObjectType(ObjectIdentity(base_oid)), lexicographicMode=False):
```

```
                if errInd or errStat:
```

```
                    break
```

```
                for vb in varbinds:
```

```
                    oid_str = str(vb[0])
```

```
                    suffix = oid_str[len(base_oid):].lstrip(".")
```

```
                    try: result[suffix] = int(vb[1])
```

```
                    except Exception: result[suffix] = str(vb[1])
```

```

except Exception:
    pass

return result

try:
    return asyncio.run(_run())
except Exception:
    return {}

```

```

def snmp_interfaces(host, community, oids, version=2):
    names = _snmp_walk(host, community, oids["ifDescr"], version)
    status = _snmp_walk(host, community, oids["ifOperStatus"], version)
    in_err = _snmp_walk(host, community, oids["ifInErrors"], version)
    out_err = _snmp_walk(host, community, oids["ifOutErrors"], version)
    in_dis = _snmp_walk(host, community, oids["ifInDiscards"], version)
    out_dis = _snmp_walk(host, community, oids["ifOutDiscards"], version)

    last_chg = _snmp_walk(host, community, oids["ifLastChange"], version) if "ifLastChange"
    in oids else {}

    status_map = {1:"up",2:"down",3:"testing",4:"unknown",5:"dormant",
                  6:"notPresent",7:"lowerLayerDown"}

    ifaces = {}

    for idx, name in names.items():
        ifaces[idx] = {
            "name": str(name),
            "status": status_map.get(status.get(idx, 4), "unknown"),
            "in_errors": int(in_err.get(idx, 0) or 0),

```

```

        "out_errors": int(out_err.get(idx, 0) or 0),
        "in_discards": int(in_dis.get(idx, 0) or 0),
        "out_discards": int(out_dis.get(idx, 0) or 0),
        "last_change_ticks": int(last_chg.get(idx, 0) or 0),
    }
    return ifaces

```

```

def snmp_system(host, community, oids, version=2):
    out = {}
    scalars = {}
    for k in ("sysUpTime", "cpmCPUTotal1min", "cpmCPUTotal5min"):
        if k in oids:
            scalars[k] = oids[k]
    if scalars:
        res = _snmp_get(host, community, list(scalars.values()), version)
        for k, oid in scalars.items():
            if oid in res:
                out[k] = res[oid]
    if "ciscoMemoryPoolUsed" in oids and "ciscoMemoryPoolFree" in oids:
        used = _snmp_walk(host, community, oids["ciscoMemoryPoolUsed"], version)
        free = _snmp_walk(host, community, oids["ciscoMemoryPoolFree"], version)
        if used and free:
            u = next(iter(used.values())); f = next(iter(free.values()))
            if isinstance(u, int) and isinstance(f, int) and (u + f) > 0:
                out["mem_used_pct"] = round(100.0 * u / (u + f), 2)

```

```
if "ciscoEnvMonSupplyState" in oids:
    pwr = _snmp_walk(host, community, oids["ciscoEnvMonSupplyState"], version)
    if pwr:
        out["power_ok"] = all(s == 1 for s in pwr.values())
return out
```

```
def collect(device_cfg, oids, use_snmp=True, use_ssh_fallback=False):
```

```
    ts = datetime.utcnow().isoformat(timespec="seconds")
```

```
    host = device_cfg["host"]; name = device_cfg["name"]
```

```
    snap = {"ts": ts, "device": name, "interfaces": {}}
```

```
    addrs = [host] + list(device_cfg.get("alt_hosts", []))
```

```
    reach_results = {a: ping(a) for a in addrs}
```

```
    primary = reach_results[host]
```

```
    if primary["reachable"]:
```

```
        snap.update(primary)
```

```
    else:
```

```
        responding = [r for r in reach_results.values() if r["reachable"]]
```

```
        if responding:
```

```
            best = min(responding, key=lambda r: r["latency_ms"]) or
```